Design using System Verilog

Assignment_2

- **Toshiv Kumar, 1024060266, 3F32, ECE**

Aim: Design and Simulation of 2's complement adder/subtractor.

Perform the following activities:

1. Write the System Verilog Code for a full adder and check its syntax errors, if any. Also, check its operation.

Code:

Design file:

```
`timescale 1ns / 1ps


// Toshiv Kumar(NAME) , 1024060266(Roll Number)
module question_1(
        input logic cin, // We can use logic or bit
        input logic a,
        input logic b,
        output logic sum,
        output logic cout
    );

    assign sum = a ^ b ^ cin;
    assign cout = a&b | b&cin | cin&a;

endmodule
```

TB file:

```
`timescale 1ns / 1ps



module TB_question1();
	logic cin_tb;
	logic a_tb;
	logic b_tb;
	logic sum_tb;
	logic cout_tb;

	question_1 dut (cin_tb, a_tb, b_tb, sum_tb, cout_tb);

	initial begin
	a_tb = 0;
	b_tb = 0;
	cin_tb = 0;

	#5 a_tb = 1;
	b_tb = 1;

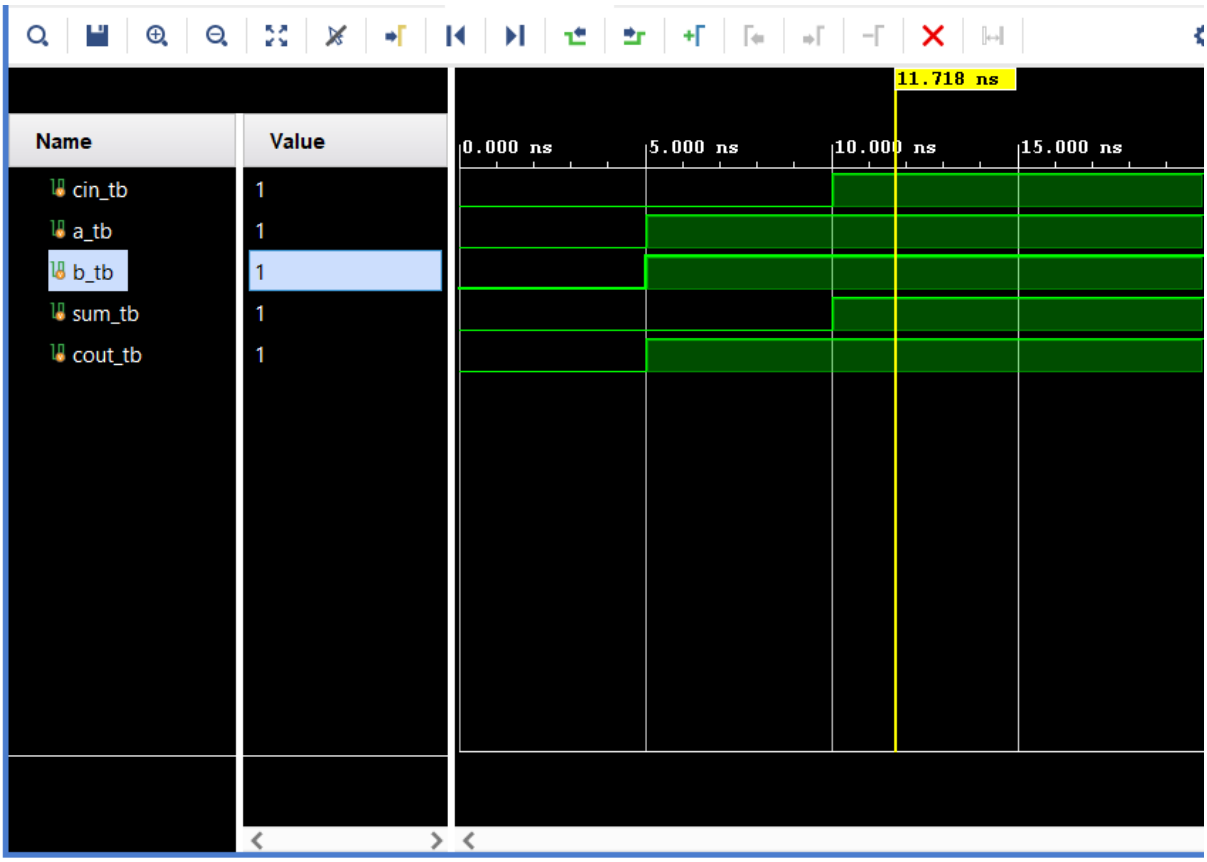
	# 5
	a_tb = 1;
	b_tb = 1;
	cin_tb = 1;

	#10 $finish;
```

end

endmodule

Output/Screenshot:



2. Write the System Verilog Code for n-bit ripple carry adder (RCA) using full adder components. Verify the operation of RCA.

Code:

Design file:

`timescale 1ns / 1ps

// Toshiv Kumar(NAME) , 1024060266(Roll Number)

module full_adder(

    input logic a,

    input logic b,

    input logic cin,

```
    output logic sum,
    output logic cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (b & cin) | (cin & a);
endmodule

module question_2 #(parameter N = 4)(
    input  logic [N-1:0] a,
    input  logic [N-1:0] b,
    input  logic         cin,
    output logic [N-1:0] sum,
    output logic         cout
);
    logic [N:0] c;
    assign c[0] = cin;
    assign cout = c[N];

    genvar i;
    generate
        for (i = 0; i < N; i = i + 1) begin : fa_chain
            full_adder fa (
                .a(a[i]),
                .b(b[i]),
                .cin(c[i]),
                .sum(sum[i]),
                .cout(c[i+1])
            );
        end
```

```
    endgenerate
endmodule
```

TB:

```
`timescale 1ns / 1ps

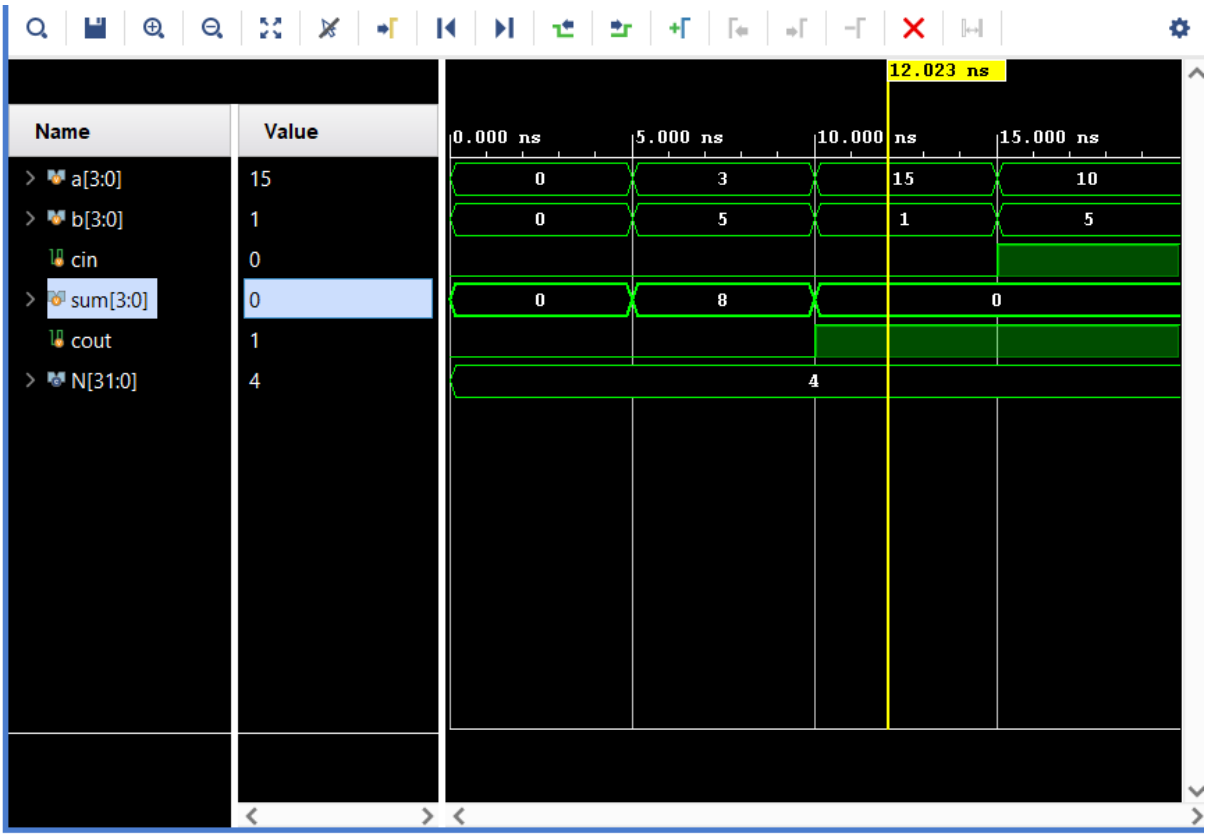

module TB_question_2;
    parameter N = 4;
    logic [N-1:0] a;
    logic [N-1:0] b;
    logic cin;
    logic [N-1:0] sum;
    logic cout;

    question_2 #(N) dut (
        .a(a),
        .b(b),
        .cin(cin),
        .sum(sum),
        .cout(cout)
    );

    initial begin
        a = 0; b = 0; cin = 0;
        #5 a = 4'b0011; b = 4'b0101; cin = 0;
        #5 a = 4'b1111; b = 4'b0001; cin = 0;
        #5 a = 4'b1010; b = 4'b0101; cin = 1;
        #5 $finish;
    end
```

```
    initial begin
        $monitor("time=%0t a=%b b=%b cin=%b sum=%b cout=%b", $time, a, b, cin, sum, cout);
    end
endmodule
```

Output/Screenshot:



3. Write the System Verilog Code for n-bit adder_subtractor using RCA. Verify the operation of adder_subtractor.

Hint: Use the approach for the addition or subtraction of signed numbers using 2's complement

approach.

Code:

Design:

```
`timescale 1ns / 1ps

// Toshiv Kumar(NAME) , 1024060266(Roll Number)

module full_adder(
    input logic a,
    input logic b,
    input logic cin,
    output logic sum,
    output logic cout
);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (b & cin) | (cin & a);
endmodule

module rca #(parameter n = 4)(
    input logic [n-1:0] a,
    input logic [n-1:0] b,
    input logic cin,
    output logic [n-1:0] sum,
    output logic cout
);
    logic [n:0] c;
    assign c[0] = cin;
    assign cout = c[n];

    genvar i;
    generate
        for (i = 0; i < n; i = i + 1) begin : fa_chain
```

```
            full_adder fa(
                .a(a[i]),
                .b(b[i]),
                .cin(c[i]),
                .sum(sum[i]),
                .cout(c[i+1])
            );
        end
    endgenerate
endmodule

module question_3 #(parameter n = 4)(
    input logic [n-1:0] a,
    input logic [n-1:0] b,
    input logic s,
    output logic [n-1:0] result,
    output logic cout
);
    logic [n-1:0] bx;

    genvar i;
    generate
        for (i = 0; i < n; i = i + 1) begin : xor_chain
            xor g1(bx[i], b[i], s);
        end
    endgenerate

    rca #(n) u1(
        .a(a),
```

```
        .b(bx),
        .cin(s),
        .sum(result),
        .cout(cout)
    );
endmodule
```

TB:

```
`timescale 1ns / 1ps

module TB_question_3;
    logic [3:0] a;
    logic [3:0] b;
    logic s;
    logic [3:0] result;
    logic cout;

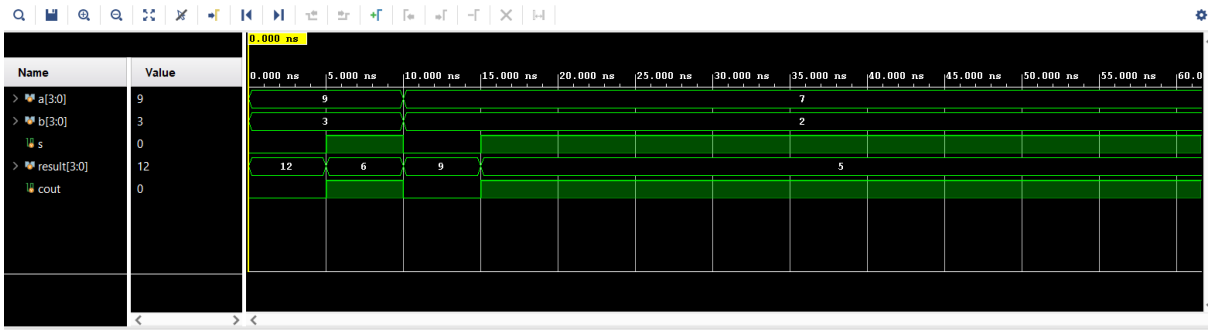
    adder_subtractor #(4) dut(
        .a(a),
        .b(b),
        .s(s),
        .result(result),
        .cout(cout)
    );

    initial begin
        a = 4'd9; b = 4'd3; s = 0;
        #5 a = 4'd9; b = 4'd3; s = 1;
        #5 a = 4'd7; b = 4'd2; s = 0;
```

```
    #5 a = 4'd7; b = 4'd2; s = 1;
    #5 $finish;
  end

  initial begin
    $monitor("time=%0t a=%0d b=%0d s=%b result=%0d cout=%b", $time, a, b, s, result, cout);
  end
endmodule
```

Output/Screenshot:




```
Time resolution is 1 ps
time=0 a=9 b=3 s=0 result=12 cout=0
time=5000 a=9 b=3 s=1 result=6 cout=1
time=10000 a=7 b=2 s=0 result=9 cout=0
time=15000 a=7 b=2 s=1 result=5 cout=1
$finish called at time : 20 ns : File "C:/VIP/3rdYear_System_verilog_VIP/Assignment_2.srcs/sim_1/new/TB_question_3.sv" Line 2
relaunch_sim: Time (s): cpu = 00:00:03 ; elapsed = 00:00:18 . Memory (MB): peak = 1262.688 ; gain = 0.000
```

Learnings from this Experiment:

There are many things that I learned from this experiment. First of all, the logic keyword and its benefits. I learned about the $monitor keyword and how to use it. I learned how to declare and use the generate keyword. At last, I learned how to create a testbench module to verify my design.